

JavaScript Interview Preparation

Beginner → Intermediate | Software Developer Interviews

A practical, compressed study guide you can get through in a single focused day — see the hour-by-hour plan in Section 23.

STUDY ROADMAP

- **Level 1** — Fundamentals: syntax, variables, data types, operators, conditions, loops, functions
- **Level 2** — Core JavaScript: arrays, objects, strings, scope, hoisting, this, callbacks, higher-order functions
- **Level 3** — Modern JavaScript: let/const, destructuring, spread/rest, modules, optional chaining, nullish coalescing
- **Level 4** — Async JavaScript: event loop, promises, async/await, fetch, error handling
- **Level 5** — Browser/DOM: DOM selection, events, forms, storage, event delegation
- **Level 6** — Intermediate topics: closures, prototypes, classes, debouncing/throttling, shallow vs deep copy
- **Level 7** — Interview practice: output questions, coding questions, rapid-fire theory, mini project discussion

1. How to Use This Document

Study in order. Do not memorize definitions alone: after each topic, write the small examples yourself and explain the concept aloud as if you were answering an interviewer.

2. What Interviewers Usually Expect

Area	Priority	What you should be able to do
Variables & types	★★★★★	Explain var/let/const, primitive/reference values, type coercion.
Functions & scope	★★★★★	Explain function types, scope, closures, callbacks.
Arrays & objects	★★★★★	Use map/filter/reduce, object operations, destructuring.
Async JavaScript	★★★★★	Explain promises, async/await, event loop and API calls.
DOM & events	★★★★☆	Manipulate elements and explain event bubbling/delegation.
this / prototypes	★★★★☆	Explain this binding and prototype inheritance.
Modern syntax	★★★★☆	Use ES6+ features naturally.
Coding problems	★★★★★	Solve small array/string/object problems cleanly.

3. JavaScript Fundamentals

3.1 What is JavaScript?

JavaScript is a high-level, dynamically typed programming language used to build interactive web applications. It runs in browsers and also outside browsers through runtimes such as Node.js.

Q: Is JavaScript the same as Java?

A: No. They are different languages. Java is class-based and commonly compiled to bytecode for the JVM; JavaScript is dynamically typed and commonly executed by a JavaScript engine such as V8.

3.2 Variables: var, let, const

```
var x = 10;
let age = 21;
const pi = 3.14;

age = 22; // allowed
// pi = 4; // TypeError
```

- `var` is function-scoped and can be redeclared.
- `let` is block-scoped and can be reassigned but not redeclared in the same scope.
- `const` is block-scoped and cannot be reassigned.
- Prefer `const` by default; use `let` when reassignment is required. Avoid `var` in modern code.

Q: Can a const object be changed?

A: Yes. `const` prevents reassignment of the variable binding, not mutation of the object's properties.

3.3 Data Types

- Primitive: string, number, bigint, boolean, undefined, symbol, null.
- Non-primitive/reference: objects, including arrays and functions.
- `typeof null` returns `'object'` due to a historical JavaScript behavior.

```
let name = "Siva";    // string
let marks = 90;       // number
let passed = true;    // boolean
let value;            // undefined
let empty = null;     // null
let nums = [10, 20, 30]; // object/array
let user = { name: "Siva" }; // object
```

3.4 Operators and Equality

Q: What is the difference between == and ===?

A: `==` allows type coercion before comparison; `===` checks value and type without that coercion. In interviews

and production code, `===` is generally preferred.

```
5 == "5" // true
5 === "5" // false
```

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `**`
- Comparison: `>`, `<`, `>=`, `<=`, `==`, `===`, `!=`, `!==`
- Logical: `&&`, `||`, `!`
- Nullish coalescing: `??`
- Ternary: `condition ? value1 : value2`

3.5 Truthy and Falsy

Falsy values include `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, and `NaN`. Most other values are truthy, including `[]` and `{}`.

```
if ([]) console.log("runs"); // runs
if ({}) console.log("runs"); // runs
```

3.6 Conditions and Loops

```
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}

for (let i = 0; i < 5; i++) {
  console.log(i);
}

for (const item of [10, 20, 30]) {
  console.log(item);
}
```

4. Functions — Very Important

4.1 Function Declaration, Expression, Arrow Function

```
function add(a, b) {  
  return a + b;  
}  
  
const multiply = function(a, b) {  
  return a * b;  
};  
  
const subtract = (a, b) => a - b;
```

- Functions are first-class values: they can be stored in variables, passed as arguments, and returned from other functions.
- Arrow functions have lexical `this` and do not create their own `arguments` object.
- Function declarations are hoisted differently from function expressions.

Q: What exactly does "hoisted differently" mean here?

A: A `function add(){}` declaration is hoisted completely — both the name and the function body are available before the line is reached, so you can call it earlier in the file. A function expression like `const multiply = function(){}` only hoists the `const multiply` binding (into the temporal dead zone), not the assignment — calling `multiply()` before that line throws a `ReferenceError`. This trips people up because it looks like ordinary hoisting but behaves like `let / const` hoisting instead.

```
console.log(add(2, 3));      // 5 – works, declarations are fully hoisted  
console.log(multiply(2, 3)); // ReferenceError: Cannot access 'multiply' before initialization  
  
function add(a, b) { return a + b; }  
const multiply = function(a, b) { return a * b; };
```

Q: What is the arguments object, and why don't arrow functions have one?

A: Inside a regular function, `arguments` is an array-like object holding every argument passed in, regardless of the named parameters — useful before rest parameters existed. Arrow functions don't get their own `arguments`; if you reference it inside an arrow function, JavaScript looks it up in the enclosing non-arrow function's scope instead (or throws if there isn't one). This is a direct consequence of arrow functions not creating their own execution context — they inherit `this`, `arguments`, and `super` from where they're defined.

```
function showArgs() {  
  console.log(arguments); // [1, 2, 3] (array-like, not a real array)  
}  
showArgs(1, 2, 3);  
  
// Modern preference: use rest parameters instead – they ARE real arrays  
function showArgsModern(...args) {  
  console.log(args); // [1, 2, 3], with full array methods  
}
```

4.2 Parameters, Return, Default, Rest

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
function sum(...numbers) {  
  return numbers.reduce((total, n) => total + n, 0);  
}  
  
console.log(sum(1, 2, 3, 4)); // 10
```

Q: Can you mix default parameters and rest parameters?

A: Yes, but rest must always come last since it collects everything remaining. A parameter with a default is treated as optional; if the caller omits it (or passes `undefined` explicitly), the default value is used. Passing `null` does NOT trigger the default — only `undefined` does.

```
function createUser(name, role = "guest", ...permissions) {  
  return { name, role, permissions };  
}  
  
createUser("Siva"); // role: "guest", permissions: []  
createUser("Siva", undefined, "read"); // role: "guest" (undefined triggers default)  
createUser("Siva", null, "read"); // role: null (default NOT triggered)
```

4.2b IIFE — Immediately Invoked Function Expression

An IIFE runs as soon as it's defined, creating an isolated scope. Before ES6 modules and block scope, this was the standard way to avoid polluting the global scope.

```
(function() {  
  const secret = "hidden";  
  console.log("runs immediately");  
})();  
  
// Arrow-function IIFE  
(() => {  
  console.log("also runs immediately");  
})();
```

Q: Why would you still use an IIFE today?

A: Less often now that modules and block-scoped `let` / `const` exist, but it's still useful for one-off setup code, wrapping library code to avoid leaking globals, or creating a private scope for a self-contained script tag that isn't a module.

4.3 Higher-Order Functions and Callbacks

A higher-order function accepts a function, returns a function, or both. A callback is a function passed to another function to be executed later or as part of its operation.

```
function processUser(name, callback) {  
  callback(name);  
}
```

```
}  
  
processUser("Siva", function(name) {  
  console.log("Hello " + name);  
});
```

Q: What is a callback?

A: A callback is a function passed to another function so that it can be invoked at an appropriate time.

Q: What's the difference between a synchronous and an asynchronous callback?

A: A synchronous callback runs immediately, during the execution of the outer function — like the callback passed to `Array.prototype.map`. An asynchronous callback runs later, after some operation completes — like a `setTimeout` callback or a Node.js file-read callback. The distinction matters because code written after a call with an async callback keeps running immediately; it does not wait for the callback.

```
// Synchronous callback — runs during map(), in order  
[1, 2, 3].map(n => console.log(n));  
  
// Asynchronous callback — scheduled for later  
setTimeout(() => console.log("later"), 1000);  
console.log("this logs first");
```

5. Scope, Hoisting and Closures

5.1 Scope

- Global scope: accessible from the global environment.
- Function scope: variables declared with `var` inside a function.
- Block scope: variables declared with `let/const` inside `{}`.
- Lexical scope: a function can access variables from the scope where it was defined.

Q: What's a concrete example of `var`'s function scope causing a bug that `let/const` would prevent?

A: A `var` declared inside an `if`-block or `for`-loop "leaks" out to the whole function, which can silently overwrite an outer variable of the same name. `let/const` are confined to the nearest enclosing block, so the same code either behaves correctly or throws a clear error instead of silently misbehaving.

```
function demo() {  
  if (true) {  
    var x = 10; // leaks out of the if-block  
    let y = 20; // stays inside the if-block  
  }  
  console.log(x); // 10 - accessible outside the block  
  console.log(y); // ReferenceError - y is not defined here  
}
```

5.2 Hoisting

Hoisting is the behavior in which declarations are processed before execution of the surrounding code. `var` declarations are initialized with `undefined`; `let` and `const` are in the temporal dead zone until their declaration is reached.

```
console.log(a); // undefined  
var a = 10;  
  
// console.log(b); // ReferenceError  
let b = 20;
```

Q: Does JavaScript move the code physically to the top?

A: No. Hoisting is a useful mental model for how declarations are created/initialized by the execution environment; the source code itself is not literally moved.

Q: What exactly is the Temporal Dead Zone (TDZ)?

A: The TDZ is the span of code between the start of a block and the point where a `let/const` variable is actually declared. The variable technically exists in that scope (which is why it's not treated as a global), but accessing it throws a `ReferenceError` rather than returning `undefined`. This is intentional — it catches bugs that `var`'s "silently undefined" behavior would hide.

```
{  
  // TDZ for `score` starts here  
  console.log(typeof score); // ReferenceError (NOT "undefined")  
}
```

```

    let score = 100;           // TDZ ends here
  }

  // Compare with var – no TDZ, just undefined:
  console.log(typeof missing); // "undefined", no error
  var missing;

```

5.3 Closures — Frequently Asked

A closure occurs when a function retains access to variables from its lexical outer scope even after the outer function has finished executing.

```

function counter() {
  let count = 0;

  return function() {
    count++;
    return count;
  };
}

const c = counter();
console.log(c()); // 1
console.log(c()); // 2

```

Q: Why are closures useful?

A: They are useful for data privacy, factories, callbacks, event handlers, memoization, and maintaining state between function calls.

Q: The classic closure trap — what does this loop print, and why?

A: With `var`, all three callbacks share the same single function-scoped variable `i`. By the time the timers fire (after the loop has already finished), `i` is 3, so all three log 3. With `let`, each loop iteration gets its own fresh block-scoped binding of `i`, so each callback closes over a different value and logs 0, 1, 2 as expected. This is one of the most commonly asked closure questions in interviews.

```

// With var – prints 3, 3, 3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

// With let – prints 0, 1, 2
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

// Fixing the var version without switching to let,
// using an IIFE to create a new scope per iteration:
for (var i = 0; i < 3; i++) {
  (function(captured) {
    setTimeout(() => console.log(captured), 0);
  })(i);
}

```

Q: Can you show closures used for memoization?

A: Memoization caches the result of an expensive function call keyed by its arguments, so repeated calls with the same input return instantly. The cache object lives in the closure, private to the returned function.

```
function memoize(fn) {
  const cache = {};
  return function(n) {
    if (n in cache) {
      console.log("from cache");
      return cache[n];
    }
    const result = fn(n);
    cache[n] = result;
    return result;
  };
}

const slowSquare = n => { for (let i = 0; i < 1e6; i++) {} return n * n; };
const fastSquare = memoize(slowSquare);

fastSquare(5); // computed
fastSquare(5); // "from cache" - instant
```

6. Arrays and Array Methods

Array methods are extremely common in interviews. Know what each returns and whether it mutates the original array.

Method	Purpose	Returns
map()	Transform every element	New array
filter()	Keep matching elements	New array
reduce()	Accumulate into one result	Single value
forEach()	Run code for each element	undefined
find()	Find first matching element	Element / undefined
some()	Check if at least one matches	boolean
every()	Check if all match	boolean
includes()	Check whether value exists	boolean
sort()	Sort elements	Same array (mutates)
slice()	Copy a portion	New array
splice()	Add/remove elements	Mutates array

```
const nums = [1, 2, 3, 4, 5];

const doubled = nums.map(n => n * 2);
const even = nums.filter(n => n % 2 === 0);
const total = nums.reduce((sum, n) => sum + n, 0);

console.log(doubled); // [2,4,6,8,10]
console.log(even);    // [2,4]
console.log(total);   // 15
```

Q: What is the difference between map and forEach?

A: map creates and returns a new array from the transformed elements. forEach is normally used for side effects and returns undefined.

Q: Why does [10, 2, 1].sort() give a "wrong" result, and how do you fix it?

A: By default, sort() converts elements to strings and compares them in UTF-16 order — so numbers sort as if they were text ("10" comes before "2" because "1" < "2" as characters). To sort numbers correctly, pass a comparator function that returns a negative, zero, or positive number.

```
const values = [10, 2, 1, 21];

console.log(values.sort());
// [1, 10, 2, 21] - wrong! sorted as strings
```

```
console.log(values.sort((a, b) => a - b));  
// [1, 2, 10, 21] – correct ascending numeric sort  
  
console.log(values.sort((a, b) => b - a));  
// [21, 10, 2, 1] – descending
```

Q: Can you write `reduce()` to solve a real problem, not just sum numbers?

A: Yes — `reduce()` is general-purpose: it can build objects, group data, count occurrences, or flatten arrays. The key is understanding the accumulator carries forward between each call.

```
// Count occurrences of each value  
const votes = ["yes", "no", "yes", "yes", "no"];  
const tally = votes.reduce((acc, vote) => {  
  acc[vote] = (acc[vote] || 0) + 1;  
  return acc;  
}, {});  
console.log(tally); // { yes: 3, no: 2 }
```

7. Objects

```
const user = {
  name: "Siva",
  age: 21,
  greet() {
    console.log("Hello " + this.name);
  }
};

console.log(user.name);
console.log(user["age"]);
user.greet();
```

- Use dot notation when the property name is known.
- Use bracket notation for dynamic property names.
- `Object.keys(obj)` returns keys.
- `Object.values(obj)` returns values.
- `Object.entries(obj)` returns [key, value] pairs.

7.1 Destructuring

```
const user = { name: "Siva", age: 21 };
const { name, age } = user;

const nums = [10, 20, 30];
const [first, second] = nums;
```

7.2 Spread vs Rest

```
// Spread: expands values
const a = [1, 2];
const b = [...a, 3, 4];

// Rest: collects remaining values
function addAll(...values) {
  return values.reduce((s, x) => s + x, 0);
}
```

Q: Does spread create a deep copy?

A: No. For nested objects/arrays, spread creates a shallow copy. Nested references are still shared.

8. this Keyword

this refers to the execution context determined by how a function is called. Its value can change depending on the call site. Arrow functions are special: they capture this lexically from the surrounding scope.

```
const person = {
  name: "Siva",
  normal() {
    console.log(this.name);
  },
  arrow: () => {
    console.log(this.name);
  }
};

person.normal(); // Siva
// person.arrow() does not get person as this
```

- `obj.method()` → this is generally `obj`.
- Regular function called plainly in strict mode → this is undefined.
- `new Constructor()` → this refers to the new instance.
- `call`/`apply`/`bind` can explicitly control this for regular functions.
- Arrow functions do not bind their own this.

Q: What's the actual difference between call, apply, and bind?

A: All three let you explicitly set what `this` refers to inside a function. `call` invokes the function immediately with arguments listed individually. `apply` invokes it immediately with arguments passed as an array. `bind` does NOT invoke the function — it returns a new function with `this` permanently locked in, to be called later.

```
function introduce(greeting, punctuation) {
  return `${greeting}, I'm ${this.name}${punctuation}`;
}

const user = { name: "Siva" };

introduce.call(user, "Hi", "!"); // "Hi, I'm Siva!" – args listed
introduce.apply(user, ["Hi", "!"]); // "Hi, I'm Siva!" – args as array

const boundIntroduce = introduce.bind(user);
boundIntroduce("Hello", "."); // "Hello, I'm Siva." – called later
```

Q: Common real-world case: why does `this` become undefined when you pass an object method as a callback?

A: Passing `obj.method` as a value (e.g. to `setTimeout` or as an event handler) detaches it from `obj` — only the function itself is passed, not the object it was called on. When it's later invoked plainly, `this` is no longer `obj`. Fix it with `.bind(obj)`, an arrow-function wrapper, or by calling it as `() => obj.method()`.

```
const timer = {
  seconds: 0,
  start() {
    // setTimeout(this.tick, 1000); // BUG: this.tick loses its "this"
  }
}
```

```
    setTimeout(this.tick.bind(this), 1000); // fixed with bind
    setTimeout(() => this.tick(), 1000);    // fixed with arrow wrapper
  },
  tick() {
    this.seconds++;
  }
};
```

9. Modern JavaScript (ES6+)

- let and const
- Arrow functions
- Template literals
- Destructuring
- Spread/rest
- Default parameters
- Classes
- Modules: import/export
- Optional chaining: obj?.property
- Nullish coalescing: value ?? fallback
- Promises and async/await

```
const user = {
  profile: {
    name: "Siva"
  }
};

console.log(user.profile?.name);
console.log(user.address?.city ?? "Unknown");
```

Q: Difference between || and ??

A: || uses any falsy value as a reason to use the fallback, while ?? uses the fallback only when the left side is null or undefined.

10. Asynchronous JavaScript

10.1 Synchronous vs Asynchronous

Synchronous code executes step by step. Asynchronous operations allow work such as timers, network requests, or I/O to complete later while JavaScript continues handling other work.

10.2 Promise

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) resolve("Done");
  else reject("Failed");
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error))
  .finally(() => console.log("Finished"));
```

Q: Promise states?

A: A promise starts pending and eventually becomes fulfilled or rejected. Once settled, it does not transition to another state.

10.3 async/await

```
async function getData() {
  try {
    const response = await fetch("https://example.com/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Q: Does await block the entire JavaScript application?

A: No. await pauses the async function's continuation until the promise settles; it does not freeze the whole JavaScript runtime.

Q: If you await two independent requests one after another, is that a performance mistake?

A: Yes, usually. Awaiting sequentially means the second request doesn't even start until the first finishes — their time adds up. If the requests don't depend on each other, start both first and then await them together (or use Promise.all) so they run concurrently and you only wait as long as the slower one.

```
// Slower – sequential, total time = time(A) + time(B)
async function sequential() {
```

```

const a = await fetchA();
const b = await fetchB();
return [a, b];
}

// Faster – concurrent, total time = max(time(A), time(B))
async function concurrent() {
  const [a, b] = await Promise.all([fetchA(), fetchB()]);
  return [a, b];
}

```

10.4 Promise.all vs Promise.allSettled

- **Promise.all**: resolves when all promises fulfill; rejects as soon as one rejects.
- **Promise.allSettled**: waits for all to settle and gives the status/result of each.
- **Promise.race**: settles when the first input promise settles.
- **Promise.any**: fulfills when the first input fulfills; rejects only when all reject.

Q: If one request fails, Promise.all rejects immediately — how do you still get the results of the successful ones?

A: Use Promise.allSettled instead. It never short-circuits — it always waits for every promise and returns an array of `{status, value}` or `{status, reason}` objects, so you can separate the succeeded requests from the failed ones after the fact.

```

const results = await Promise.allSettled([fetchA(), fetchB(), fetchC()]);

results.forEach((result, i) => {
  if (result.status === "fulfilled") {
    console.log(`Request ${i} succeeded:`, result.value);
  } else {
    console.log(`Request ${i} failed:`, result.reason);
  }
});

```

10.5 Event Loop — Interview Explanation

JavaScript executes synchronous code on the call stack. Asynchronous APIs such as timers and network operations are handled by the host environment. When callbacks are ready, they wait in queues. The event loop coordinates when queued work can be moved onto the call stack. Promise reactions use the microtask queue, which is processed before the next task such as a timer callback.

```

console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");

// Output:
// A
// D
// C
// B

```

Q: Why is C printed before B?

A: The promise callback is a microtask. After the current synchronous code finishes, microtasks are processed before the timer task.

Q: What's the difference between the microtask queue and the macrotask (task) queue, precisely?

A: Promise callbacks (.then/.catch/.finally), queueMicrotask, and async/await continuations go into the microtask queue. setTimeout, setInterval, and I/O callbacks go into the macrotask queue. After each single macrotask finishes, the event loop drains the ENTIRE microtask queue — including any new microtasks scheduled during that drain — before it renders or picks up the next macrotask. This is why microtasks can "starve" the event loop if they keep scheduling more of themselves.

Queue	Examples	When it runs
Call stack	Regular synchronous code	Immediately, top to bottom
Microtask queue	Promise .then/.catch/.finally, async/await, queueMicrotask	Fully drained after each macrotask, before the next render or macrotask
Macrotask (task) queue	setTimeout, setInterval, I/O, UI events	One task per event-loop tick, after microtasks are drained

Q: Harder example — can you trace the exact output?

A: Work through it queue by queue: synchronous code runs first top to bottom (1, 4), then all microtasks drain fully (2, 3 — note both .then callbacks fire before the timeout even though they were scheduled after the first console.log), and only then does the macrotask (5) run.

```
console.log(1);

setTimeout(() => console.log(5), 0);

Promise.resolve()
  .then(() => console.log(2))
  .then(() => console.log(3));

console.log(4);

// Output: 1, 4, 2, 3, 5
```

11. DOM and Browser Basics

11.1 Selecting Elements

```
const title = document.getElementById("title");
const button = document.querySelector("#saveButton");
const items = document.querySelectorAll(".item");
```

Q: getElementById vs querySelector?

A: getElementById directly selects an element by its id. querySelector accepts a CSS selector and returns the first matching element.

11.2 innerHTML vs innerText vs textContent

- innerHTML reads/writes HTML markup.
- innerText focuses on rendered text and is affected by CSS/layout.
- textContent gets/sets text content without interpreting it as HTML.

11.3 Events and Event Listeners

```
const button = document.querySelector("#save");

button.addEventListener("click", (event) => {
  console.log("Clicked");
});
```

11.4 Event Bubbling and Delegation

Events commonly bubble from the target element toward ancestors. Event delegation uses one listener on a parent to handle events from matching child elements. This is useful for dynamic lists and reduces the number of listeners.

```
document.querySelector("#list").addEventListener("click", (event) => {
  if (event.target.matches(".delete")) {
    console.log("Delete item");
  }
});
```

Q: What's the difference between stopPropagation and preventDefault?

A: `stopPropagation()` stops the event from continuing to bubble (or capture) to ancestor elements — other listeners further up the tree won't fire. `preventDefault()` stops the browser's default action for that event (like a link navigating, a form submitting, or a checkbox toggling) but does NOT stop the event from bubbling. They solve different problems and are often used together but are not interchangeable.

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the page from reloading
  // event.stopPropagation(); // would additionally stop bubbling to ancestors
  submitViaFetch(form);
});
```

```
});
```

12. Browser Storage

- localStorage persists until explicitly cleared.
- sessionStorage generally lasts for the page session/tab lifecycle.
- Both store strings, so objects commonly require JSON.stringify/JSON.parse.
- Do not casually store highly sensitive secrets in browser storage; understand the security implications before doing so.

```
const user = { name: "Siva" };
localStorage.setItem("user", JSON.stringify(user));

const saved = JSON.parse(localStorage.getItem("user"));
console.log(saved.name);
```

13. Prototypes and Classes

JavaScript uses prototype-based inheritance. Objects can access properties/methods through their prototype chain. `class` syntax provides a more familiar way to define constructor functions and methods, but it is built on JavaScript's prototype system.

```
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hello ${this.name}`;
  }
}

const p = new Person("Siva");
console.log(p.greet());
```

Q: What is a prototype?

A: A prototype is an object from which another object can inherit properties and methods through the prototype chain.

Q: How does the prototype chain actually work under the hood — can you show it without class syntax?

A: Every object has an internal link (accessible via `Object.getPrototypeOf(obj)` or the legacy `obj.__proto__`) to another object — its prototype. When you access a property, JavaScript checks the object itself first, and if not found there, walks up the prototype chain until it finds the property or reaches `null`. `class` syntax is essentially syntactic sugar over exactly this mechanism.

```
// The class version above is equivalent to this:
function PersonOldStyle(name) {
  this.name = name;
}

PersonOldStyle.prototype.greet = function() {
  return `Hello ${this.name}`;
};

const p2 = new PersonOldStyle("Siva");
console.log(p2.greet()); // found on PersonOldStyle.prototype
console.log(p2.hasOwnProperty("name")); // true — own property
console.log(p2.hasOwnProperty("greet")); // false — inherited via the chain
```

Q: What's the difference between `prototype` and `__proto__`?

A: `prototype` is a property that exists only on functions/classes — it's the object that will become the prototype of instances created with `new`. `__proto__` (or `Object.getPrototypeOf()`) exists on every object instance and points to that object's actual prototype. So `p2.__proto__ === PersonOldStyle.prototype` is true.

14. Shallow Copy vs Deep Copy

```
const original = {
  name: "Siva",
  address: { city: "Karur" }
};

const shallow = { ...original };
shallow.address.city = "Chennai";

console.log(original.address.city); // Chennai
```

The nested address object is still shared. For structured data, `structuredClone(value)` can create a deep clone for supported values. JSON stringify/parse is an older workaround with important limitations.

Q: What exactly breaks with `JSON.parse(JSON.stringify(obj))` as a deep-clone technique?

A: It silently drops or mangles anything JSON can't represent: functions and `undefined` values are removed, `Date` objects become plain strings, `Map` / `Set` become empty objects, and it throws on circular references. `structuredClone()` (built into modern browsers and Node 17+) handles all of these correctly and is the preferred approach today.

```
const data = {
  when: new Date(),
  greet: () => "hi",
  count: undefined,
  tags: new Set(["a", "b"])
};

const jsonClone = JSON.parse(JSON.stringify(data));
console.log(jsonClone);
// { when: "2026-08-29T..." (string, not Date), tags: {} } – greet and count are gone

const realClone = structuredClone(data);
// realClone.when is a real Date, realClone.tags is a real Set
// (structuredClone still can't clone functions – greet is dropped there too)
```

15. Debouncing and Throttling

Q: What is debouncing?

A: Debouncing delays execution until a specified period has passed without another call. It is useful for search boxes and reducing rapid API requests.

Q: What is throttling?

A: Throttling limits execution to at most a chosen frequency during a period. It is useful for scroll, resize, or mouse-move handlers.

Q: What's the concrete behavioral difference between them in a real scenario?

A: For a search-as-you-type box firing on every keystroke: debouncing waits until the user pauses typing for, say, 300ms, then fires once — ideal because you only care about the final query. Throttling would instead fire at a steady rate WHILE the user types, which wastes requests on intermediate, incomplete queries. For a scroll handler that updates a "scroll progress" bar, throttling is right — you want regular updates throughout the scroll, not just one at the end.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

function throttle(fn, limit) {
  let inCooldown = false;
  return function(...args) {
    if (inCooldown) return;
    fn.apply(this, args);
    inCooldown = true;
    setTimeout(() => { inCooldown = false; }, limit);
  };
}

// Usage
searchInput.addEventListener("input", debounce(fetchResults, 300));
window.addEventListener("scroll", throttle(updateProgressBar, 200));
```

16. Error Handling

```
try {
  riskyOperation();
} catch (error) {
  console.error(error.message);
} finally {
  console.log("Cleanup");
}
```

- throw creates an exception.
- try/catch handles synchronous exceptions and can also handle errors around awaited promises.
- Always handle rejected promises appropriately.
- Do not silently swallow errors.

Q: How do you create and use a custom error type?

A: Extend the built-in Error class. This preserves the stack trace and instanceof checks while letting you attach extra context (like an HTTP status code) and a distinct name, which callers can check to decide how to handle it.

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

function validateAge(age) {
  if (age < 0) throw new ValidationError("Age cannot be negative", "age");
}

try {
  validateAge(-5);
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Invalid field "${error.field}": ${error.message}`);
  } else {
    throw error; // re-throw anything we don't recognize
  }
}
```

Q: Does try/catch catch errors from setTimeout or unawaited promises inside it?

A: No. try/catch only catches errors thrown synchronously during that block's execution. A setTimeout callback or a promise that isn't awaited runs later, outside the try block's call stack, so its errors escape uncaught. You must handle those separately — with their own try/catch (if awaited) or a .catch() handler.

```
try {
  setTimeout(() => { throw new Error("boom"); }, 100);
} catch (e) {
  console.log("never runs – the error escapes uncaught");
}
```

17. Modules

```
// math.js
export function add(a, b) {
  return a + b;
}

// app.js
import { add } from "./math.js";

console.log(add(2, 3));
```

Q: Why use modules?

A: Modules organize code, control what is exported, avoid unnecessary global variables, and improve maintainability and reuse.

18. Common Interview Traps

Trap	Remember
<code>typeof null</code>	returns "object".
<code>[] == false</code>	true because of coercion; avoid relying on loose equality.
<code>[] === []</code>	false because they are different object references.
<code>const object</code>	properties can still be mutated.
<code>map()</code>	returns a new array.
<code>forEach()</code>	does not return the transformed array.
<code>sort()</code>	mutates the array and, by default, compares values as strings.
<code>spread</code>	shallow copy, not deep copy.
<code>arrow this</code>	lexical this; it does not create its own this.
Promise callback vs timer	promise microtasks normally run before timer tasks after the current stack clears.

19. Must-Practice JavaScript Coding Questions

1. Reverse a string without using `reverse()`.
2. Check whether a string is a palindrome.
3. Find the largest and second-largest number in an array.
4. Remove duplicates from an array.
5. Count the frequency of each character in a string.
6. Find the first non-repeating character.
7. Find duplicate elements in an array.
8. Move all zeros to the end of an array.
9. Find the missing number from $1..n$.
10. Flatten a one-level nested array.
11. Group an array of objects by a property.
12. Sort an array of objects by age/name.
13. Find the sum of numbers using `reduce()`.
14. Filter users whose age is greater than 18.
15. Implement a debounce function.
16. Implement a simple counter using closure.
17. Fetch data from an API using `async/await` and handle errors.
18. Create a todo list using DOM events and event delegation.

20. Mini Coding Examples

Remove duplicates

```
const nums = [1, 2, 2, 3, 4, 4];
const unique = [...new Set(nums)];
console.log(unique); // [1,2,3,4]
```

Frequency counter

```
function frequency(str) {
  const map = {};

  for (const ch of str) {
    map[ch] = (map[ch] || 0) + 1;
  }

  return map;
}

console.log(frequency("hello"));
```

Group objects

```
const users = [
  { name: "A", role: "dev" },
  { name: "B", role: "test" },
  { name: "C", role: "dev" }
];

const grouped = users.reduce((acc, user) => {
  (acc[user.role] ??= []).push(user);
  return acc;
}, {});
```

21. 30 Rapid-Fire Interview Questions

1. What is JavaScript?
2. What is dynamic typing?
3. var vs let vs const?
4. Primitive vs reference values?
5. == vs ===?
6. What are truthy and falsy values?
7. What is hoisting?
8. What is the temporal dead zone?
9. What is scope?
10. What is a closure?
11. What is a callback?
12. What is a higher-order function?
13. map vs filter vs reduce?
14. forEach vs map?
15. slice vs splice?
16. What does spread syntax do?
17. What is destructuring?
18. What is this?
19. Arrow function vs regular function?
20. What is a Promise?
21. Promise states?
22. async/await?
23. What is the event loop?
24. Microtask vs task?
25. What is DOM?
26. Event bubbling?
27. Event delegation?
28. localStorage vs sessionStorage?
29. Prototype vs class?
30. Shallow copy vs deep copy?

22. How to Answer in an Interview

Use a simple 4-step structure:

- **Definition** — give a one- or two-sentence definition.
- **Difference/working** — explain the key mechanism or contrast.
- **Example** — write a tiny code example.
- **Use case** — mention when you would use it in a real project.

Example

If asked "What is a Promise?", start with the definition, explain pending/fulfilled/rejected, show `.then/.catch` or `async/await`, and mention API calls as a common use case.

23. One-Day JavaScript Interview Crash Plan

This is a compressed, single-day version of the study plan. It assumes roughly 10 focused hours with short breaks. Skim first, then come back and do the code examples by hand — don't just read them.

Time Block	Focus
Hour 1	Fundamentals: var/let/const, data types, operators, equality, truthy/falsy, conditions, loops (Sections 3)
Hour 2	Functions: declarations vs expressions vs arrows, parameters, default/rest, higher-order functions, callbacks (Section 4)
Hour 3	Scope, hoisting, and closures — write the counter() example yourself from memory (Section 5)
Hour 4	Arrays & objects: map/filter/reduce, destructuring, spread vs rest (Sections 6-7)
Break	15-20 minute break — this is where retention actually happens
Hour 5	this keyword and modern ES6+ syntax: optional chaining, nullish coalescing (Sections 8-9)
Hour 6	Async JavaScript: promises, async/await, Promise.all/allSettled/race/any, event loop (Section 10) — this is the highest-priority topic, don't rush it
Hour 7	DOM & browser: selecting elements, events, bubbling/delegation, storage (Sections 11-12)
Hour 8	Prototypes/classes, shallow vs deep copy, debounce/throttle, error handling, modules (Sections 13-17)
Break	15-20 minute break
Hour 9	Common interview traps (Section 18) + attempt all 18 coding questions (Section 19) — write real code, don't just read the list
Hour 10	Rapid-fire 30 questions (Section 21) out loud, using the 4-step answer structure (Section 22). Finish with the checklist (Section 24)

If you only have 3-4 hours

Prioritize in this order: closures (5.3), array methods (Section 6), promises/async/await/event loop (Section 10), this keyword (Section 8), then the rapid-fire questions (Section 21). These cover the large majority of what gets asked.

24. Final Interview Checklist

- ☐ I can explain var, let, const without memorizing.
- ☐ I can explain primitive/reference values and type coercion.
- ☐ I can use map, filter, reduce, find, some, every, sort, slice, splice.
- ☐ I can explain scope, hoisting, TDZ, closure and lexical scope.
- ☐ I can explain this and arrow-function behavior.
- ☐ I can explain promises and async/await.
- ☐ I can explain the event loop at a high level.
- ☐ I can manipulate the DOM and handle events.
- ☐ I know event bubbling and delegation.
- ☐ I understand shallow vs deep copy.
- ☐ I can explain prototypes and classes.
- ☐ I can solve basic array/string/object coding questions.
- ☐ I can explain my code clearly while coding.

Goal: Understand the concept → write a small example → explain it aloud → solve one coding problem.